SONORA

Aïda GOUMBLE, Rachel DIGNA, Eva DOUMBE NGANGUE, Clara SION-SAUNIER

1. Vision Fonctionnelle 1
1.1 Motivation 2
1.2 Profils Utilisateurs 2
1.3 Backlog - User Story 3
1.3.1 Authentification 3
1.3.2 Catalogue Musical 3
1.3.3 Streaming Audio 4
1.3.4 Playlists 4
2. Organisation et Planning 5
2.1 Planning des Sprints 5
2.2 Organisation 5
3. Architecture et Stack technique 6
3.1 Architecture logicielle 6
3.1.1 Vue d'ensemble 6
3.1.2 Schéma d'architecture 7
3.1.3 Découpage des packages Spring Boot 7
3.1.4 Modèle de données 8
3.2 Plateforme DevOps 9

1. Vision Fonctionnelle

1.1 Motivation

Sonora est une application web de streaming musical permettant aux utilisateurs de découvrir, écouter et organiser leur musique. Le projet est développé dans le cadre du module Fullstack S8 avec l'obligation d'intégrer Apache Kafka comme infrastructure de messagerie distribuée.

Ses objectifs sont :

- Proposer un catalogue musical (titres, albums, artistes) consultable et filtrable
- Permettre l'écoute en streaming de fichiers audio (MP3/WAV)
- Permettre la création de playlists manuelles et de playlists automatisées
- Introduire un twist fonctionnel : la génération automatique de playlists par mood et/ou genre
- Démontrer l'usage réel de Kafka dans un contexte de distribution d'événements

Le différenciateur principal de Sonora est la fonctionnalité de "Smart Playlist" : l'utilisateur sélectionne un ou plusieurs critères (mood, genre, année, artiste) et l'application génère automatiquement une playlist personnalisée à partir du catalogue disponible. Cette fonctionnalité s'appuie sur l'historique d'écoute de l'utilisateur (consommé via Kafka) pour affiner les résultats au fil du temps.

1.2 Profils Utilisateurs

Voici trois Persona :

Persona 1 - L'artiste en devenir

Lucas, 22 ans, musicien en herbe

Il arrive sur Sonora avec l'intention de faire découvrir sa musique a d'autres personnes. Il peut consulter le catalogue (titres, artistes, albums) et aussi uploader ses productions. Son besoin : Agrandir son public.

Persona 2 - L'Utilisateur Standard

Inès, 27 ans, amatrice de musique, utilise Spotify mais veut quelque chose de plus personnalisé

Elle se connecte, parcourt le catalogue, écoute des titres en streaming, crée des playlists manuelles et utilise la Smart Playlist pour ses sessions de travail (mood FOCUS + genre JAZZ). Elle aime aussi mettre des titres en favoris pour les retrouver. Son besoin : une expérience fluide avec un lecteur audio réactif.

Persona 3 - L'Administrateur

Marc, 35 ans, gestionnaire du catalogue musical

Il a accès aux fonctionnalités d'administration : uploader des fichiers MP3/WAV, ajouter/modifier/supprimer des titres, albums et artistes. Il ne s'occupe pas des playlists des utilisateurs. Son besoin : une interface d'admin claire, un upload rapide avec traitement asynchrone des métadonnées.

1.3 Backlog - User Story

1.3.1 Authentification

ID	Rôle	Je veux...	Priorité
US-01	Visiteur	M'inscrire avec un email et un mot de passe pour accéder à l'application	Must Have
US-02	Utilisateur	Me connecter et me déconnecter de mon compte	Must Have
US-03	Utilisateur	Accéder à mon profil et voir mon historique d'écoute	Should Have

1.3.2 Catalogue Musical

ID	Rôle	Je veux...	Priorité
US-04	Utilisateur	Parcourir la liste de tous les titres disponibles	Must Have
US-05	Utilisateur	Voir le détail d'un titre (artiste, album, genre, durée, mood)	Must Have
US-06	Utilisateur	Rechercher un titre, un artiste ou un album par mot-clé	Must Have
US-07	Utilisateur	Filtrer les titres par genre, année et/ou mood	Must Have
US-08	Utilisateur	Ajouter, modifier et supprimer un titre du catalogue (titre qui a été ajouter par lui)	Must Have

US-09	Admin	Ajouter, modifier et supprimer n’importe quel titre du catalogue	Should Have

1.3.3 Streaming Audio

ID	Rôle	Je veux...	Priorité
US-10	Utilisateur	Écouter un titre en streaming directement dans l'interface	Must Have
US-11	Utilisateur	Mettre en pause, reprendre et avancer dans un titre	Must Have
US-12	Utilisateur	Uploader un fichier MP3 ou WAV pour l'ajouter au catalogue	Must Have
US-13	Admin	Uploader un fichier MP3 ou WAV pour l'ajouter au catalogue	Should Have

1.3.4 Playlists

ID	Rôle	Je veux...	Priorité
US-14	Utilisateur	Créer une playlist manuelle et y ajouter des titres	Must Have
US-15	Utilisateur	Modifier le nom et le contenu d'une playlist et ajouter une image d’illustration	Should Have
US-16	Utilisateur	Générer une Smart Playlist en sélectionnant un mood et/ou un genre	Must Have
US-17	Utilisateur	Sauvegarder une Smart Playlist pour la réécouter plus tard	Should Have
US-18	Utilisateur	Ajouter un titre à mes favoris	Should Have

2. Organisation et Planning

2.1 Planning des Sprints

Sprint	Période	Objectifs	Livrable clé
Sprint 0	jusqu'au 28/02	Mise en place DevOps : repo Git, branches, pipeline CI (build + tests), docker-compose Kafka + PostgreSQL + Spring Boot vide. Rédaction du document de conception.	Document de conception + CI fonctionnelle
Sprint 1	16/03 → 22/03	CRUD Track/Album/Artist. Upload fichier + streaming HTTP Range. Intégration Kafka (track.imported). Auth JWT. Angular : catalogue, recherche, lecteur audio.	Backend core + streaming fonctionnel
Sprint 2	23/03 → 27/03	Playlists manuelles + Smart Playlists (génération par mood/genre). Favoris. Topic track.played + stats. Amélioration CI (JaCoCo). Swagger complet.	MVP fonctionnel (Milestone 2)
Sprint 3	13/04 → 24/04	Couverture tests ≥ 70%. Tests d'intégration REST. Audit sécurité. README complet. Versioning Docker 1.0. Préparation soutenance.	Release 1.0 + démo production

2.2 Organisation

Les différents tickets seront créés et assignés directement sur GitHub afin qu'on puisse suivre l'avancée des autres et catégoriser les tâches à faire selon le sprint et la nature de la tâche.

Personne	Feature	Description
DEV A	Auth + Profil utilisateur	Angular (login/register/profil), Spring Security + JWT, entités User, endpoints
DEV B	Catalogue + Recherche	Angular (pages catalogue, détail titre, recherche, filtres), CRUD Track/Album/Artist, endpoints
DEV C	Streaming + Upload	Angular (lecteur audio HTML5, player component), HTTP Range Requests, upload MP3, Kafka track.imported, consumer métadonnées
DEV D	Playlists + Smart Playlist	Angular (pages playlist, générateur Smart Playlist), entités Playlist/Favorite, Kafka track.played, stats, endpoints

3. Architecture et Stack technique

3.1 Architecture logicielle

3.1.1 Vue d'ensemble

L'application suit une architecture en couches (monolithe modulaire) côté backend, complétée par un cluster Kafka pour la gestion asynchrone des événements. Ce choix évite la complexité des microservices tout en démontrant concrètement un usage de Kafka.

Couche	Technologies
Frontend	Angular 17, TypeScript, Angular Material, HTML5 Audio API
Backend	Java 21, Spring Boot 3.x, Spring Security, Spring Data JPA
API Externe	Spotify API
Messagerie	Apache Kafka 3.x, Zookeeper, Spring Kafka

Base de données	PostgreSQL 15
Fichiers audios	Stockage local sur volume Docker, streaming HTTP Range Requests
DevOps	Docker, docker-compose, GitHub Actions, JaCoCo
Documentation	Swagger / OpenAPI 3

3.1.2 Schéma d'architecture

```
[Angular Frontend]
      ⇕ REST HTTP / multipart
[Spring Boot API]
   ├── Module Auth (Spring Security + JWT)
   ├── Module Catalogue (Track / Album / Artist)
   ├── Module Streaming (Range Requests)
   ├── Module Playlist (manuelle + Smart)
   ├── Module Spotify Client
   └── Module Événements (Kafka Producers)
      ⇕ Kafka
[Kafka Consumers]
   ├── Consumer track.played → Stats + Smart Playlist
   └── Consumer track.imported → Extraction métadonnées + Appel Spotify API
      ⇕
[PostgreSQL]   [Volume Docker - fichiers audio]
```

3.1.3 Découpage des packages Spring Boot

- controller - endpoints REST (TrackController, PlaylistController, AuthController...)
- service - logique métier (TrackService, SmartPlaylistService, StreamingService...)
- repository - interfaces JPA (TrackRepository, PlaylistRepository...)
- model / entity - entités JPA (Track, Album, Artist, Playlist, User...)
- dto - objets de transfert et validation (TrackDto, PlaylistRequestDto...)
- kafka - producers et consumers (TrackEventProducer, TrackPlayedConsumer...)
- security - configuration Spring Security, JWT filter
- exception - handlers globaux et exceptions métier

3.1.4 Modèle de données

Entité	Champs principaux	Relations
User	id, email, passwordHash, role (USER/ADMIN), createdAt	1 User → N Playlists, N Favorites
Artist	id, name, bio, imageUrl	1 Artist → N Albums, N Tracks
Album	id, title, releaseYear, coverUrl, artistId	1 Album → N Tracks, N Artist
Track	id, title, duration, genre, mood, filePath, albumId, artistId	N Tracks ↔ N Playlists
Playlist	id, name, type (MANUAL/SMART), userId, createdAt	1 User → N Playlists, N Tracks
PlayHistory	id, userId, trackId, playedAt, durationListened	Source des stats et Smart Playlists
Favorite	id, userId, trackId, createdAt	Table de liaison User/Track

Les ENUMS:

- Genre: POP, ROCK, JAZZ, CLASSICAL, ELECTRONIC, HIP_HOP, REGGAE, METAL, RNB, OTHER
- Mood: HAPPY, SAD, CHILL, FOCUS, HYPE, ROMANTIC, WORKOUT, ANGRY
- PlaylistType: MANUAL, SMART
- Role: USER, ADMIN

3.2 Plateforme DevOps

Nous aurons deux branches : une branche main et une branche dev. Chaque ticket fera l'objet d'une branche basée sur dev, qui sera merge sur dev. Notre workflow se déclenche sur chaque Pull Request vers `dev` ou `main`, et sur chaque push sur `dev`.

Étape 1 - Build & Test Backend

- Checkout du code
- compile + tests unitaires + JaCoCo
- Publication du rapport JaCoCo en artifact GitHub
- Le pipeline échoue si la couverture est < 70%

Étape 2 - Build Frontend

- Vérification que le build ne produit pas d'erreurs TypeScript

Étape 3 - Build Docker (uniquement sur push `dev` et `main`)

- `docker build` de l'image Spring Boot
- Tag de l'image
- Push vers GitHub Container Registry, le dépôt d'artefacts

Étape 4 - Déploiement (uniquement sur `main`)

- Tag de l'image Docker avec la version Git tag
- L'image taguée `latest` est mise à jour

Gestion des Merge Requests / Pull Requests :

- PR obligatoire pour tout merge vers `dev` ou `main`
- La CI doit être verte (build + tests passent)
- 1 reviewer minimum doit approuver
- Les commits doivent respecter la convention : `feat:`, `fix:`, `chore:`, `test: etc.`